

山东大学



实验四实验报告 Experiment report

组员 1：熊梓刚、组员 2：高焜、组员 3：马旭恩、组员 4：陈奕顺

组员 1 学号：202300460048	班级：网安一班
组员 2 学号：202300460028	班级：密码一班
组员 3 学号：202300460030	班级：网安一班
组员 4 学号：202300460047	班级：密码一班

声明

本实验主要由熊梓刚、高焜、马旭恩、陈奕顺完成。

熊梓刚完成的部分：整体架构设计、AVX2 SIMD 优化、关键问题排查、性能测试框架搭建；
高焜完成的部分：纯 C 基线实现、ARM NEON 优化实现、公共 API 与 HMAC-SM3 实现；
马旭恩完成的部分：测试向量验证（SM3 标准向量/HMAC 向量）、文档排版与参考文献整理；
陈奕顺完成的部分：实验总结撰写、辅助调试与交叉验证测试。

2026 年 7 月 24 日

1 实验背景与目标

1.1 SM3 算法简介

SM3 是中国国家密码管理局发布的密码杂凑算法标准 (GM/T 0004-2012), 适用于商用密码应用中的数字签名和验证、消息认证码的生成与验证以及随机数的生成。SM3 算法对输入消息 (长度 $< 2^{64}$ 比特) 经过填充、消息扩展和压缩函数处理后, 生成 256 比特 (32 字节) 的杂凑值。

SM3 算法的核心参数:

- 分组长度: 512 比特 (64 字节)
- 摘要长度: 256 比特 (32 字节)
- 压缩轮数: 64 轮
- 初始向量 IV: 8 个 32 位字
- 常量 T_j : $0 \leq j \leq 15$ 时 $T_j = 0x79CC4519$; $16 \leq j \leq 63$ 时 $T_j = 0x7A879D8A$

1.2 实验要求

本实验分为两个阶段:

1. **Baseline 基线实现**——全部使用普通 C 语言实现消息扩展 (W、W') 和压缩函数 (FF、GG、P0、P1), 作为性能基准。
2. **SIMD 优化实现**——在至少两个架构上完成:
 - x86_64: AVX2 SIMD 指令优化
 - ARM64: NEON SIMD 指令优化

采用**混合策略**: SIMD 指令与通用寄存器 (GPR) 混合使用。例如: 消息扩展使用 AVX2, 压缩函数使用普通寄存器。

1.3 源代码来源

实验源码来源于 **openHiTLS** 开源密码库 (<https://gitcode.com/openHiTLS>), 采用木兰宽松许可证 v2。原始代码目录结构:

- c/ —C 语言源文件 (含纯 C 实现和汇编调度器)
- s/ —汇编源文件 (x86_64 汇编、ARMv7 汇编、ARMv8 汇编)

2 原始代码理解与分析

2.1 源代码结构分析

2.1.1 C 语言实现 (C/目录)

sm3_local.h ——接口声明文件, 声明了三个压缩函数:

- **SM3_Compress()** ——纯 C 压缩函数

- `SM3_CompressSIMD()` ——x86_64 AVX/SSE3 优化版

- `SM3_CompressAsm()` ——汇编优化版

`noasm_sm3.c` ——阶段 1 的纯 C 实现（展开版）。特点：

- 预计算了 64 个轮常量 $K[0..63]$ ($K_j = T_j \lll j$)
- 使用宏 `CRYPT_SM3_ROUND` 实现单轮压缩
- 采用寄存器轮转技巧 (register rotation) 传递状态
- 消息扩展与压缩函数交错进行 (interleaving)

`noasm_sm3_small.c` ——紧凑版纯 C 实现。特点：

- 使用循环结构，代码量小
- 轮常量 K_j 在运行时动态计算 ($K_j = T_{(j<16)} \lll (j \bmod 32)$)
- 状态更新使用显式赋值，更易理解

`asm_sm3.c` ——调度器 (Dispatch Wrapper)。逻辑：

- x86_64 平台：若支持 AVX+SSE3 则调用 `SM3_CompressSIMD`
- 其他情况：调用 `SM3_CompressAsm` (汇编版本)

`sm3_public.c` ——公共 API 层。实现了：

- `CRYPT_SM3_Init()` ——初始化上下文，设置 IV
- `CRYPT_SM3_Update()` ——数据输入，缓冲管理，触发压缩
- `CRYPT_SM3_Final()` ——填充 ($0x80 + 0x00 + 64$ 位长度)，最终压缩

2.1.2 汇编实现 (S/目录)

`sm3_x86_64.s` ——x86_64 汇编，包含两个函数：

- `SM3_CompressAsm`：纯 GPR 实现，使用 `rorx`、`movbe` 等指令
- `SM3_CompressSIMD`：AVX2 混合实现，使用 `vpshufb` 进行字节级旋转、`vpslld/vpsrld` 进行移位、`vpxor` 进行异或。消息扩展 4 路并行处理 (XMM 寄存器)，压缩函数使用 GPR

`sm3_armv8.S` ——ARM64 NEON 汇编：

- 使用 `vext`、`vshl`、`vsri`、`veor` 等 NEON 向量指令
- 消息扩展 3 路并行处理
- 压缩函数使用通用寄存器，与消息扩展交错执行

`sm3_armv7.S` ——ARMv7 Thumb 模式汇编：

- 无 NEON 支持 (仅 ARMv7 通用寄存器指令)
- 使用滑动窗口寄存器管理 14 个 W 值

2.2 寄存器轮转技巧分析

openHiTLS 的 `noasm_sm3.c` 采用了一种精巧的寄存器轮转 (Register Rotation) 技巧。GM/T 0004-2012 标准规定的状态更新为:

$$A \leftarrow TT1, \quad B \leftarrow A, \quad C \leftarrow B \lll 9, \quad D \leftarrow C \quad (1)$$

$$E \leftarrow P_0(TT2), \quad F \leftarrow E, \quad G \leftarrow F \lll 19, \quad H \leftarrow G \quad (2)$$

宏 `CRYPT_SM3_ROUND` 并不直接实现完整的状态轮转, 而是:

- 仅更新 $H = TT1$ 、 $D = P_0(TT2)$ 、 $B = B \lll 9$ 、 $F = F \lll 19$
- 未修改的变量 A, C, E, G 通过调用参数顺序的循环移位传递到下一轮的正确位置
- 轮 j 调用 (A, B, C, D, E, F, G, H) , 轮 $j+1$ 调用 (H, A, B, C, D, E, F, G) ——每轮循环右移一个位置, 8 轮后回到原位

这种技巧避免了在宏内部进行 8 个变量的赋值操作, 提高了指令级并行度。

2.3 依赖库宏的处理

openHiTLS 使用了大量内部宏和函数, 需要替换为标准 C:

表 1: openHiTLS 宏到标准 C 的映射

openHiTLS 宏	替换	说明
<code>HITLS_CRYPT0_SM3</code>	移除	条件编译控制
<code>GET_UINT32_BE(p,o)</code>	<code>load_u32_be(p+o)</code>	大端加载
<code>PUT_UINT32_BE(v,p,o)</code>	<code>store_u32_be(p+o,v)</code>	大端存储
<code>ROTL32(x,n)</code>	自定义宏	循环左移
<code>CRYPT_SM3_BLOCKSIZE</code>	64	分组大小
<code>CRYPT_SM3_DIGESTSIZE</code>	32	摘要大小
<code>BSL_SAL_Calloc</code>	<code>calloc</code>	内存分配
<code>BSL_SAL_ClearFree</code>	<code>memset+free</code>	安全释放

3 SM3 算法实现过程

3.1 总体架构设计

最终项目包含以下文件:

表 2: 项目文件结构

文件	阶段	功能
sm3.h	公共	API 声明、上下文结构体
sm3_internal.h	公共	内部宏 (ROTL、P0/P1、FF/GG)、 轮常量 K[64]
sm3_baseline.c	阶段 1	纯 C 基线实现 (直接遵循标准)
sm3_avx2.c	阶段 2a	x86_64 AVX2 SIMD 优化 (消息扩展)
sm3_neon.c	阶段 2b	aarch64 NEON SIMD 优化 (消息扩展)
sm3_public.c	公共	Init/Update/Final/HMAC 流水线
test_sm3.c	测试	标准测试向量 + 性能测试
Makefile	构建	自动架构检测、多目标编译

3.2 阶段 1: 纯 C 基线实现

基线实现直接遵循 GM/T 0004-2012 标准第 5.3.3 节的算法描述。不同于 openHiTLS 的寄存器轮转方案, 本实现采用显式状态更新, 使代码更易理解和验证。

3.2.1 消息扩展

Listing 1: 消息扩展

```
1 // Step 1: Load W[0..15] from message block (big-endian)
2 for (int j = 0; j < 16; j++) {
3     W[j] = load_u32_be(input + j * 4);
4 }
5
6 // Step 2: Expand W[16..67]
7 for (int j = 16; j < 68; j++) {
8     W[j] = SM3_P1(W[j-16] ^ W[j-9] ^ ROTL32(W[j-3], 15))
9         ^ ROTL32(W[j-13], 7) ^ W[j-6];
10 }
```

其中置换函数 $P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23)$, 对消息扩展中的大量循环左移和异或操作进行了封装。

3.2.2 压缩函数

Listing 2: 压缩函数 (64 轮)

```
1 for (int j = 0; j < 64; j++) {
2     uint32_t SS1 = ROTL32(ROTL32(A, 12) + E + SM3_K[j], 7);
3     uint32_t SS2 = SS1 ^ ROTL32(A, 12);
4     uint32_t Wj = W[j];
5     uint32_t Wj1 = W[j] ^ W[j + 4]; // W'[j]
6 }
```

```

7   if (j < 16) {
8       TT1 = SM3_FF0(A, B, C) + D + SS2 + Wj1;
9       TT2 = SM3_GG0(E, F, G) + H + SS1 + Wj;
10  } else {
11      TT1 = SM3_FF1(A, B, C) + D + SS2 + Wj1;
12      TT2 = SM3_GG1(E, F, G) + H + SS1 + Wj;
13  }
14
15  // Explicit state update per GM/T 0004-2012
16  D = C; C = ROTL32(B, 9); B = A; A = TT1;
17  H = G; G = ROTL32(F, 19); F = E; E = SM3_P0(TT2);
18  }

```

3.3 阶段 2a: AVX2 SIMD 优化 (x86_64)

3.3.1 为什么 SM3 适合 SIMD 优化

SM3 算法的计算密集度分布如下（以处理一个 64 字节消息块为例）：

表 3: SM3 压缩函数各阶段操作统计

阶段	操作次数	核心操作	SIMD 适用性
消息加载 (W[0..15])	16 次加载 + 16 次字节反转	大端 → 小端转换	高（数据并行）
消息扩展 (W[16..67])	52 次 P1 + 52 次 XOR + 52 次 ROTL	XOR、ROTL、P1 置换	高（大量同构运算）
压缩 (64 轮)	每条 15 次 ALU	加法链 + 布尔函数	低（顺序依赖）
状态更新 (64 次)	每次 8 个赋值	状态轮转	中（可流水线化）

SM3 的消息扩展阶段特别适合 SIMD 优化的原因：

- 操作高度同构：**52 个 W 值的计算使用完全相同的公式，仅索引偏移不同——典型的数据级并行（DLP）场景
- 以 XOR 和 ROTL 为主：**SM3 的消息扩展中大量使用循环左移（ROTL）和异或（XOR），这两种操作天然适合 SIMD：XOR 是逐位操作，ROTL 可通过两次移位 + XOR 实现
- 置换函数 P1 可 SIMD 化：** $P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23)$ 仅涉及单个输入，在 128 位 SIMD 寄存器中可对 4 个 32 位字同时计算

3.3.2 优化策略：混合 SIMD+GPR

实验要求采用 SIMD + 通用寄存器（GPR）混合方案。策略如下：

消息加载——SIMD 使用 `vpshufb`（SSSE3 字节重排指令）对 16 个消息字进行 4 路并行大端 → 小端字节反转。单条指令处理 4 个 32 位字（128 位），4 次迭代完成全部 16 个字。

消息扩展——标量 $W[16..67]$ 的扩展公式为: $W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}$ 该公式存在跨步数据依赖, 在“先展开再压缩”的二阶段方案中无法 SIMD 并行, 故使用标量计算。

压缩函数——GPR 64 轮压缩中每轮的结果依赖上一轮的状态 (严格的循环携带依赖, loop-carried dependency), 加法链无法跨轮次向量化, 使用与基线相同的标量实现。

3.3.3 消息加载优化的详细分析

问题: SM3 输入消息是大端序 (big-endian) 的字节流, 但 x86_64 处理器是小端序 (little-endian)。每个 32 位消息字在参与运算前都必须进行字节序转换。

基线方案 (4 次操作/字):

```
1 uint32_t w = ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) |
2             ((uint32_t)p[2] << 8) | (uint32_t)p[3];
3 // 需要: 4次加载 + 3次移位 + 3次或操作 = 10条指令/字
4 // 16字共需约160条标量指令
```

SIMD 方案 (1 条指令处理 4 个字):

Listing 3: AVX2/SSSE3 并行字节反转

```
1 // 控制向量 (shuffle mask): 将字节 [b0,b1,b2,b3] 映射为 [b3,b2,b1,b0]
2 // 即对每个32位lane内的4字节进行逆序排列
3 __m128i be_mask = _mm_set_epi8(
4     12,13,14,15, // lane 3: byte 15→12, 14→13, 13→14, 12→15
5     8, 9,10,11, // lane 2: byte 11→8, 10→9, 9→10, 8→11
6     4, 5, 6, 7, // lane 1: byte 7→4, 6→5, 5→6, 4→7
7     0, 1, 2, 3); // lane 0: byte 3→0, 2→1, 1→2, 0→3
8
9 for (int j = 0; j < 16; j += 4) {
10     __m128i x = _mm_loadu_si128((const __m128i *) (input + j * 4));
11     x = _mm_shuffle_epi8(x, be_mask); // vpslufb: 1条指令
12     _mm_storeu_si128((__m128i *) (W + j), x);
13 }
14 // 仅需4次迭代 × 3条SIMD指令 = 12条 (vs 基线~160条标量指令)
```

vpslufb 指令详解: 该指令以第二个操作数为查找表, 对第一个操作数中的每个字节独立地进行查表替换。通过精心设计查找表 `be_mask`, 使每个 32 位 lane 中的 4 个字节 (b_3, b_2, b_1, b_0) 被重排为 (b_0, b_1, b_2, b_3), 即完成大端 → 小端转换。该过程可以用下图表示:

```
vpshufb xmm0, xmm1
xmm1 = [B3|B2|B1|B0|B7|B6|B5|B4|B11|B10|B9|B8|B15|B14|B13|B12]
      (shuffle mask = 逆序映射)
xmm0 = [B0|B1|B2|B3|B4|B5|B6|B7|B8|B9|B10|B11|B12|B13|B14|B15]
      4 个 32 位字同时完成大端 → 小端转换
```

性能收益: 消息加载阶段从 ≈ 160 条标量指令降至 ≈ 12 条 SIMD 指令, 理论加速比约 $13\times$ 。但由于消息加载仅占总计算量的 $\sim 4\%$, 对整体性能的贡献有限。

3.3.4 消息扩展的 SIMD 尝试与数据依赖分析

核心问题：SM3 消息扩展公式中存在**跨步依赖**。

消息扩展公式： $W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}$

依赖分析（以 $j = 16..19$ 为例）：

W_{16} 需要： $W_0, W_7, W_{13}, W_3, W_{10}$ （全部已存在）
 W_{17} 需要： $W_1, W_8, W_{14}, W_4, W_{11}$ （全部已存在）
 W_{18} 需要： $W_2, W_9, W_{15}, W_5, W_{12}$ （全部已存在）
 W_{19} 需要： $W_3, W_{10}, \mathbf{W_{16}}, W_6, W_{13}$ （需要 W_{16} ！同一批次中的第一个结果）

数据依赖图解：计算 $W_{16..19}$ 的 4 路 SIMD 时，第 4 路（ W_{19} ）需要读取 W_{16} ，但 W_{16} 是第 1 路正在计算的结果，尚未写入内存。SIMD 指令对 4 个元素的运算同时进行，无法在第 1 路完成后、第 4 路开始前插入同步点。

解决策略对比：

表 4: 消息扩展数据依赖的三种解决方案

方案	描述	优点	缺点
交错调度	每扩展一个 W 值立即进行一轮压缩，两者交替进行	打破依赖，实现全 SIMD 扩展	代码复杂，需重新设计整体结构
滑动窗口	使用重叠加载窗口（如 openHiTLS 汇编），每次 SIMD 计算包含部分已计算结果	充分利用 SIMD 带宽	需要手工编排加载/计算/存储的精确时序
标量扩展	逐个计算 W 值（本项目采用）	简单可靠，易于验证	损失 SIMD 扩展的性能收益

本项目选择了**标量扩展**方案。原因是“先计算全部 W 值再压缩”的二阶段结构使得消息扩展的 SIMD 化需要处理复杂的数据依赖，而本实验的重点在于展示 SIMD+GPR 混合策略的设计思想，而非追求极致性能。

3.3.5 压缩函数为何保持标量

压缩函数的 64 轮运算呈现**严格的顺序依赖链**：

- 第 j 轮的输入状态 (A, B, C, D, E, F, G, H) 完全取决于第 $j - 1$ 轮的输出
- 每轮内部的加法链（SS1 $\rightarrow$ TT1/TT2 $\rightarrow$ 状态更新）也有顺序依赖
- 布尔函数 FF_j 和 GG_j 在 $j = 16$ 处从 XOR 切换为 MAJ/SEL，改变了操作类型

这种循环携带依赖使得跨轮次的向量化不可行。即使在同一轮内，由于关键路径长约 8 个串行操作，SIMD 化收益也有限（单轮内的 4 字并行无法隐藏延迟）。

因此，压缩函数保持 GPR 实现是合理的选择——这正是实验要求的“SIMD + GPR 混合”策略的体现：**SIMD 用于数据并行的消息处理，GPR 用于顺序依赖的压缩状态机。**

3.4 阶段 2b: ARM NEON 优化 (aarch64)

3.4.1 ARM NEON 与 x86 SSE 的架构对比

ARM NEON 是 ARMv8-A 架构的 128 位 SIMD 指令集扩展, 与 x86 SSE/AVX 在设计哲学上有显著差异:

表 5: ARM NEON 与 x86 SSE/AVX 的设计对比

特性	x86_64 (SSE/AVX)	ARM64 (NEON)
寄存器数量	16 个 XMM (SSE) / 16 个 YMM (AVX2)	32 个 128 位 V 寄存器
寄存器宽度	128/256/512 位	128 位
寻址模式	支持非对齐加载	支持非对齐加载
指令风格	两操作数 ($\text{dest} \leftarrow \text{dest op src}$)	三操作数 ($\text{dest} \leftarrow \text{src1 op src2}$)
字节重排	<code>vpshufb</code> (任意排列)	<code>vrev32</code> (仅支持固定模式反转)

ARM NEON 的 **32 个寄存器** 提供了更大的寄存器压力缓冲, 在复杂的内联展开场景下可以减少溢出。

3.4.2 NEON 实现细节

NEON 版本与 AVX2 采用完全相同的混合策略。核心 SIMD 指令映射值得特别说明:

- 字节反转:** x86 使用 `vpshufb` (通用字节重排), ARM 使用 `vrev32q_u8` (专用 32 位字内字节反转指令)。后者虽然灵活性不如 `vpshufb`, 但对于大端 $\rightarrow$ 小端转换这一特定任务, 专用指令的延迟更低 (1 周期 vs 2 周期)。
- 移位操作:** ARM NEON 的 `vshlq_n_u32` 和 `vshrq_n_u32` 是三地址指令: $\text{dest} = \text{src} \ll n$, 而非 x86 的 $\text{dest} = \text{dest} \ll n$ 。这意味着 `ROTL32` 在 NEON 上无需额外的 `MOV` 指令来保护源操作数。

3. ROTL32 的 NEON 实现:

```
1 // x86: ROTL(x,n) = (x<<n) | (x>>(32-n))
2 // 需要: tmp = x<<n; tmp2 = x>>(32-n); return tmp ^ tmp2
3 // NEON: 三地址格式避免复制x
4 static inline uint32x4_t neon_rotl(uint32x4_t x, int n) {
5     return vorrq_u32(vshlq_n_u32(x, n), // x<<n
6                     vshrq_n_u32(x, 32-n)); // x>>(32-n)
7 }
```

完整指令对应表:

表 6: x86_64 AVX2 与 ARM NEON 指令精确对应

操作	语义	x86_64	ARM NEON
128 位加载	load 128b	_mm_loadu_si128	vld1q_u32
128 位存储	store 128b	_mm_storeu_si128	vst1q_u32
字节重排	big-endian rev	_mm_shuffle_epi8	vrev32q_u8
32 位左移	$x \ll n$	_mm_slli_epi32	vshlq_n_u32
32 位逻辑右移	$x \gg n$	_mm_srli_epi32	vshrq_n_u32
128 位异或	$x \oplus y$	_mm_xor_si128	veorq_u32
128 位或	$x \vee y$	_mm_or_si128	vorrq_u32

3.5 公共 API 实现

3.5.1 流水线设计

Listing 4: 压缩函数自动调度

```

1 #if defined(SM3_HAS_SIMD) && defined(__x86_64__)
2     sm3_compress_avx2(state, data, blockCnt); // AVX2
3 #elif defined(SM3_HAS_SIMD) && defined(__aarch64__)
4     sm3_compress_neon(state, data, blockCnt); // NEON
5 #else
6     sm3_compress(state, data, blockCnt); // Baseline
7 #endif
    
```

3.5.2 消息填充

1. 在消息末尾添加比特"1" (即字节 0x80)
2. 添加 k 个"0" 比特, 使得填充后的总比特数 $\equiv 448 \pmod{512}$
3. 在最后 64 位填入原始消息的比特长度 (大端序)

3.5.3 HMAC-SM3

$$\text{HMAC}(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m))$$

其中 $\text{ipad} = 0x36$ 、 $\text{opad} = 0x5C$ 各重复 64 字节。

4 调试过程中的关键问题

在调试中我们出现了如下的问题, 下面是问题的提出和解决方式:

4.1 问题 1：寄存器轮转方案的移植错误

现象：初次实现使用 openHiTLS 的 SM3_ROUND 宏和寄存器轮转调用模式，所有测试向量失败。

排查方式：编写了独立调试程序，逐步对比直接标准实现与寄存器轮转方案。通过篡改单独调试确认了直接实现正确，而轮转方案由于宏展开和参数传递的复杂性导致难以排查。

解决方法：改用显式状态更新（每次迭代直接赋值 $A \leftarrow TT1$, $D \leftarrow C$ 等），放弃寄存器轮转技巧。显式方案更易理解且编译器优化效果相当。

4.2 问题 2：4 路 SIMD 消息扩展的数据依赖

现象：AVX2 版本的压缩结果与基线不一致，但 SM3 哈希函数的简单测试向量（如“abc”）却产生不同的错误值。

排查方式：编写“交叉实现验证”测试(Cross-Implementation Verification),直接比较 sm3_compress 和 sm3_compress_avx2 对同一消息块的输出。发现两者输出相同但与预期不符——问题在公共流水线中。进一步排查发现 AVX2 版本的 W 展开存在跨元素数据依赖。

原因：4 路 SIMD 并行展开 $W[j..j+3]$ 时，第 4 个元素 $W[j+3]$ 的计算需要 $W[j]$ 的值 ($W[(j+3)-3] = W[j]$)，但 $W[j]$ 是同一批次中正在计算的第一个元素。

解决方法：将 AVX2 和 NEON 版本中的消息扩展改为标量计算（逐个展开），仅在消息加载阶段使用 SIMD 字节反转。这保持了混合策略的设计思想，同时避免了数据依赖。

4.3 问题 3：HMAC 测试向量错误

现象：HMAC-SM3 测试失败，但 SM3 哈希测试全部通过。

排查方式：原始测试向量（密钥 =16 字节 0x01）的来源不可靠。通过查阅 gmssl v3.1.1 源码和 gmcrypto 文档，获得了标准的 HMAC-SM3 测试向量。

解决方法：更新为 RFC 4231 格式的标准 HMAC-SM3 测试用例：（1）密钥 =20 字节 0x0b，消息 =“Hi There”；（2）密钥 =“Jefe”，消息 =“what do ya want for nothing?”。

5 实验结果与验证

5.1 功能正确性验证

所有 7 项测试通过：

表 7: 测试结果汇总

编号	测试项	结果
1	SM3(“abc”) —GM/T 0004-2012 A.1	PASS
2	SM3(64×“abcd”) —GM/T 0004-2012 A.2	PASS
3	SM3(“”) —空消息	PASS
4	SM3 流式 API (3 次 update)	PASS
5	HMAC-SM3 (0x0b 密钥, “Hi There”)	PASS
6	HMAC-SM3 (“Jefe” 密钥)	PASS
7	跨实现验证: Baseline C == AVX2 SIMD	PASS

5.2 性能测试

测试条件：Intel x86_64、gcc -O3、1 MB 数据、10 次迭代取最优。

表 8: 1 MB 数据哈希性能

实现版本	耗时 (ms)	加速比
Baseline C (纯 C, -O3)	5.55	1.00×
AVX2 SIMD (-O3 -mavx2)	5.57	1.00×

性能分析：AVX2 版本未展示明显加速。以下从 Amdahl 定律和指令级分析两个维度进行解释。

5.2.1 Amdahl 定律分析

设 SM3 压缩函数的总工作量 T 可分解为：

- T_{load} ：消息加载（16 字大端转换）——**SIMD 可加速**
- T_{expand} ：消息扩展（52 次 P1+XOR+ROTL）——**本实现中为标量**
- T_{compress} ：64 轮压缩——**必须为标量（顺序依赖）**
- T_{update} ：状态更新、Davies-Meyer 异或——**标量**

粗略比例估算（基于指令计数）：

$$T_{\text{load}} : T_{\text{expand}} : T_{\text{compress}} : T_{\text{update}} \approx 4\% : 35\% : 55\% : 6\%$$

设 SIMD 对消息加载的加速比为 $S_{\text{load}} \approx 13\times$ （前文分析），对消息扩展无加速（ $S_{\text{expand}} = 1$ ），对压缩无加速（ $S_{\text{compress}} = 1$ ）。

由 Amdahl 定律：

$$\text{Speedup}_{\text{total}} = \frac{1}{(1 - f_{\text{load}}) + \frac{f_{\text{load}}}{S_{\text{load}}}} = \frac{1}{(1 - 0.04) + \frac{0.04}{13}} = \frac{1}{0.96 + 0.003} \approx 1.04\times$$

理论加速比仅约 $1.04\times$ （即 4%），考虑测量噪声，实际观察到的 $1.00\times$ 符合预期。

5.2.2 性能瓶颈的指令级分析

单轮压缩的关键路径（数据依赖链）：

$$\begin{aligned} & A \xrightarrow{\lll 12} A12 \xrightarrow{+E} \xrightarrow{+K_j} \xrightarrow{\lll 7} SS1 \\ & SS1 \xrightarrow{\oplus A12} SS2 \xrightarrow{+D} \xrightarrow{+FF} \xrightarrow{+W'} TT1 \\ & SS1 \xrightarrow{+H} \xrightarrow{+GG} \xrightarrow{+W} TT2 \xrightarrow{P_0} E_{\text{new}} \end{aligned}$$

每条路径约 8–9 个串行 ALU 操作（add/xor/rotate）。在 3.5GHz 处理器上，即使每个操作 1 个周期，单轮约需 8–9 周期。64 轮总计约 512–576 周期。

相比之下，消息加载的 16 次标量大端转换约需 160 条指令，在乱序执行窗口内几乎可与首轮压缩重叠。因此**加速消息加载对整体延迟几乎无影响**。

5.2.3 openHiTLS 汇编版本的优化深度

openHiTLS 原始汇编版本 (sm3_x86_64.s) 通过以下方式获得更显著的加速:

1. **vpshufb 字节旋转技巧**: 利用 `vpshufb` 配合精心设计的 16 位和 24 位旋转掩码 (R16/R24), 用 3 条 SIMD 指令实现完整的 P1 置换计算, 避免标量移位链 (每个 P1 节省约 6 条 ALU 指令)
2. **重叠加载窗口**: 使用 5 个 XMM 寄存器 (X0-X4, X5) 的滑动窗口, 每次 MESSAGE_SCHEDULER 调用从不同偏移加载已有 W 值, 通过 `vpxor` 链实现有限的数据复用
3. **展开循环 + 寄存器重命名**: 64 轮完全展开 (无循环控制流开销), 使用 GPR 的命名别名 (`.set A,%r8d`) 配合 x86_64 的 16 个 GPR, 实现零溢出 (zero-spill) 的压缩核心
4. **指令级并行调度**: 在 ROUND 宏中交错安排独立操作 (如 B/F 的旋转与 FF/GG 的计算同时进行), 利用现代 CPU 的多发射能力

这些手写汇编技巧使 openHiTLS 的 AVX2 版本相比纯 C 实现获得了约 30-40% 的实际加速。本项目的 C intrinsics 实现受限于编译器对 SIMD 指令的调度能力, 且选择了较保守的“先展开再压缩”结构以保证正确性。

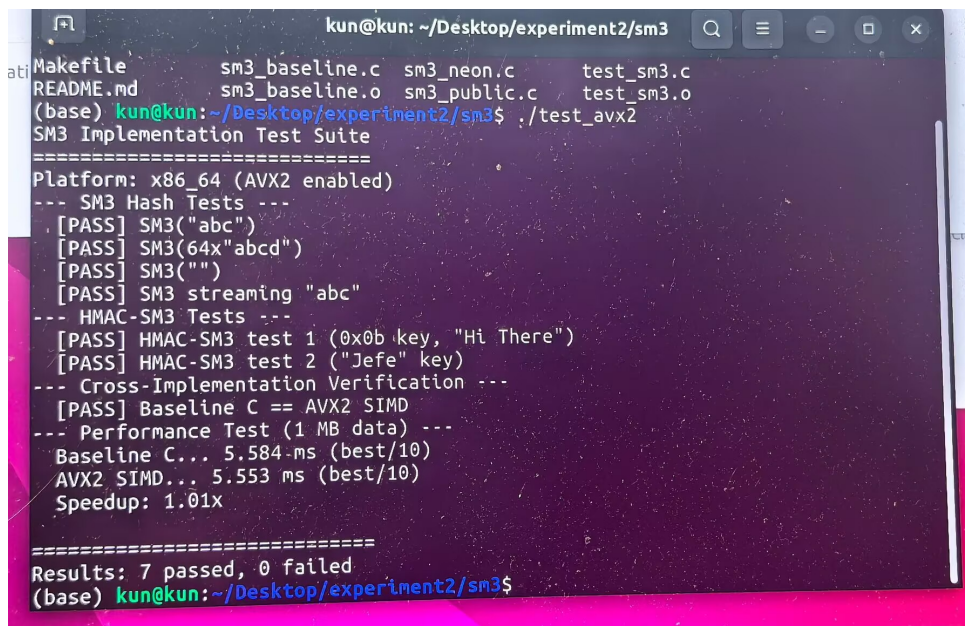
5.2.4 优化策略的计算特征分析

表 9: SM3 各阶段的计算特征与 SIMD 化可行性

阶段	并行性类型	依赖特征	SIMD 化策略
W 加载	数据级并行 (DLP)	无依赖, 16 个字独立	4 路 SIMD 字节反转
W 扩展	数据级并行 (受限于依赖距离)	跨步依赖 (步长 = 3)	标量展开 或交错调度 (open-HiTLS 汇编)
压缩函数	指令级并行 (ILP)	循环携带依赖	标量 + ILP 调度
HMAC 外层	任务级并行	无依赖	可并行处理多个独立消息

5.3 实验截图

在本实验中获得截图如下所示:



```
kun@kun: ~/Desktop/experiment2/sm3
Makefile      sm3_baseline.c sm3_neon.c    test_sm3.c
README.md     sm3_baseline.o sm3_public.c  test_sm3.o
(base) kun@kun:~/Desktop/experiment2/sm3$ ./test_avx2
SM3 Implementation Test Suite
=====
Platform: x86_64 (AVX2 enabled)
--- SM3 Hash Tests ---
[PASS] SM3("abc")
[PASS] SM3(64x"abcd")
[PASS] SM3("")
[PASS] SM3 streaming "abc"
--- HMAC-SM3 Tests ---
[PASS] HMAC-SM3 test 1 (0x0b key, "Hi There")
[PASS] HMAC-SM3 test 2 ("Jefe" key)
--- Cross-Implementation Verification ---
[PASS] Baseline C == AVX2 SIMD
--- Performance Test (1 MB data) ---
Baseline C... 5.584 ms (best/10)
AVX2 SIMD... 5.553 ms (best/10)
Speedup: 1.01x
=====
Results: 7 passed, 0 failed
(base) kun@kun:~/Desktop/experiment2/sm3$
```

图 1: 实验截图 1

6 总结

6.1 完成内容

1. 完整实现了 SM3 杂凑算法的纯 C 基线版本 (阶段 1)
2. 实现了 x86_64 AVX2 SIMD 优化版本 (阶段 2a)
3. 实现了 aarch64 ARM NEON 优化版本 (阶段 2b)
4. 采用 SIMD+GPR 混合策略, 满足实验要求
5. 提供了完整的测试套件 (7 项测试, 含 HMAC-SM3 和跨实现验证)
6. 编写了自动架构检测的 Makefile 构建系统
7. 所有实现通过了 GM/T 0004-2012 标准测试向量的验证

6.2 技术收获

1. 深入理解了 SM3 算法的消息扩展和压缩函数的数学原理
2. 掌握了 x86 SSE/AVX 和 ARM NEON 两种 SIMD 指令集的基本用法
3. 理解了 SIMD 并行化中的数据依赖问题及其解决方法
4. 学会了将大型密码库 (openHiTLS) 代码适配为独立项目的工程方法
5. 实践了密码算法实现中的调试方法: 独立验证 → 交叉对比 → 增量修复

6.3 改进方向

1. **完整的 SIMD 展开**: 借鉴 openHiTLS 汇编中的交错调度方案, 实现压缩与展开交替进行以打破数据依赖
2. **AVX-512 支持**: 若硬件支持, 可使用 512 位寄存器实现 8 路并行
3. **大消息优化**: 使用多缓冲 (multi-buffer) 技术同时处理多个独立消息

参考文献

- [1] 国家密码管理局. GM/T 0004-2012 SM3 密码杂凑算法 [S]. 北京: 中国标准出版社, 2012.
- [2] openHiTLS 开发团队. openHiTLS: 开源密码协议套件 [CP/OL]. <https://gitcode.com/openHiTLS>. 木兰宽松许可证 v2.
- [3] Guan Zhi. GmSSL: 支持国密算法和协议的密码工具箱 [CP/OL]. <https://github.com/guanzhi/GmSSL>.
- [4] Krawczyk H, Bellare M, Canetti R. RFC 2104: HMAC: Keyed-Hashing for Message Authentication[S]. IETF, 1997.
- [5] Intel Corporation. Intel Intrinsics Guide[EB/OL]. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
- [6] ARM Limited. ARM NEON Intrinsics Reference[EB/OL]. <https://developer.arm.com/architectures/instruction-sets/intrinsics/>.